PRD to PR Playbook

How a Confluence PRD becomes a reviewed pull request with test evidence, using Claude Code or Cursor, across the Airflow data repo, the micro-frontends and the frontend shell. Everything described here exists in the `prd-to-pr-rule-skills` repository and was tested. Section 18 lists what still needs checking in your environment.

Owner Engineering Platform **Source** github.com/amarkum/prd-to-pr-rule-skills

Supersedes the draft "AI-Native Engineering Implementation Plan" (see section 2)

`5` plugins, one repo for Claude Code and Cursor | `14` skills | `5` subagents | `5` enforcing hooks | `77` unit tests, Python 3.9 and 3.13 | `3` behavioural evals, all passing

CONTENTS

1	The recommendation	2	Changes to the draft plan	3	The workflow and its gates
4	What goes where	5	One builder, three repos	6	The PRD gate
7	Context sources	8	Specs	9	Credible evidence
10	Compounding knowledge	11	Setups to try	12	Claude Code features
13	Cursor features	14	Running it in an enterprise	15	Rollout and metrics
16	What others do	17	What doesn't work	18	What was verified

1

The recommendation

1. **Use the agent runtimes you already have as the platform.** Claude Code and Cursor already provide sandboxing, permissions, subagents, hooks, MCP connectors and headless CI. Package the process as one versioned plugin repository that both tools install. Build a custom orchestrator later, and only if volume or cross-repo coordination demands it.

2. **Three human gates, enforced by code.** The PRD is ready (the product owner has answered the questions), the spec is approved (by the tech lead), and the PR is merged (by code owners). Agents do the work between the gates. Hooks stop them from skipping a gate or approving their own spec.

3. **One builder, specialised per repo.** Each repo declares its stack, commands and contracts in `.ai/repo.yaml`, and stack skills load automatically by file path. The same builder writes a DAG in the data repo and a component in the micro-frontend, following each repo's conventions.

4 **Evidence is computed, not claimed.** Tests are named after acceptance criteria. A script proves that each new test fails without the change. CI re-runs everything at the PR's commit, hashes and attests the artifacts, and posts the verdict on the PR.

5 **Knowledge compounds through pull requests.** Every lesson becomes a reviewed entry with an owner and an expiry date. A lesson that recurs becomes a rule, and then an automated check.

6 **Start small and measure.** Spend two weeks on Setup B (section 11) with one repo per stack. Compare lead time, rework and escaped defects with the last 20 comparable tickets before automating any triggers.

2

Changes to the draft plan

The draft plan has the right instincts: staged pilot, approval gates, untrusted input, no automatic merge, an evidence bundle and a definition of done. It invests in infrastructure before proving the workflow, and leaves the artifacts undefined: where rules live, how they reach developers, what makes evidence credible. This playbook keeps the instincts and makes each part concrete.

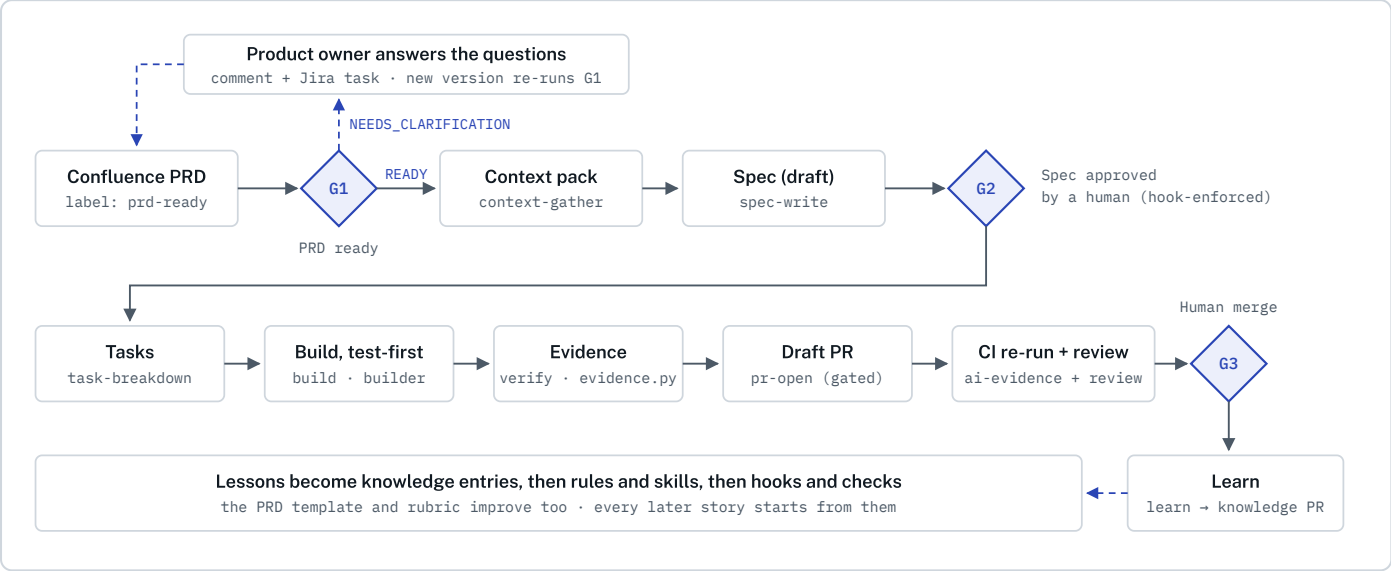
TOPIC	DRAFT PLAN	THIS PLAYBOOK	WHY
Build path	16 weeks; orchestrator, durable workflow engine and six agent services before scaling	A plugin on Claude Code and Cursor from week 1; Jira statuses hold the workflow state; an orchestrator later, if needed	The runtimes already do sandboxing, permissions, subagents, hooks and headless CI. The first real PR arrives in weeks, not months.
PRD validation	"Completeness validator"	Deterministic lint (15 sections, placeholders, Given/When/Then, vague terms, numbers, open questions) plus a rubric-based LLM review; questions go to the product owner with suggested defaults; the gate re-runs on each new page version	The product owner gets consistent feedback they can act on in one pass.
Repo knowledge	Repository manifests, architecture map	`.ai/repo.yaml` read by hooks, scripts, agents and CI; `AGENTS.md` for human-readable facts; schema snapshots in git	One file answers "how do I build, test and verify this repo" for people and machines.
Agents	Six agent services	Five subagents and fourteen skills inside one runtime; the analysts are read-only through tool restrictions	Cheaper to change, and separation of duties without new infrastructure.
Evidence	Evidence bundle in the PR	Evidence protocol: AC-named tests, fail-to-pass against the merge-base, changed-line coverage, a hashed manifest, a CI re-run at the head SHA, attestation	Green CI can hide weak tests: 15–28% of "passing" patches in public benchmarks were wrong (UTBoost).
Learning loop	"Curated reusable solution library"	Knowledge entries with an owner and a review date, a promotion ladder to rules and checks, a weekly staleness audit	Repeated review comments turn into checks instead of wiki pages.
Multi-repo	Avoid one agent with write access to every repo	Kept: one session, branch, PR and permission set per repo; contracts tested at each seam	Least privilege, and each PR reviewable on its own.

Kept unchanged: approvals for migrations, auth and infrastructure changes; treating PRD, ticket and log text as untrusted; no automatic merge or deploy during the pilot; the metrics and go/no-go design; the definition of done.

3

The workflow and its gates

Run the whole flow with `/sdlc-core:prd-to-pr <PRD link or Jira key>`. Each stage is also a skill you can run on its own. State lives in git (`specs/<KEY>/`) and in Jira, so any session or any teammate can resume, for example with `/sdlc-core:prd-to-pr PAY-412 build`.



Agents do the work in the boxes. People own the diamonds. G1 and G2 are enforced by the PRD gate and the edit hook; G3 by branch protection, the PR evidence gate and CI.

STAGE	SKILL	WRITES	EXIT CONDITION	JIRA STATUS
PRD gate	`prd-review`	`prd.md` , `prd-review.md/.json`	`READY` , or the product owner gets questions and the flow stops	Needs Clarification / Ready for Spec
Context	`context-gather`	`context.md`	Every claim cites a file, query, ticket or URL	Spec in Progress
Spec	`spec-write`	`spec.md` (draft)	`G2` a human sets `status: approved`	Spec Review → Ready for Build
Plan	`task-breakdown`	`tasks.md`	Every in-scope AC maps to a task in a named repo	In Progress
Build	`build` + `builder`	Tests, code, one commit per task	All tasks ticked; local checks green; 3 attempts per task, then escalate	In Progress
Verify	`verify` + `qa-verifier`	`.ai/evidence/`	`PASS` for the committed HEAD; 2 repair rounds, then escalate	In Progress
Draft PR	`pr-open`	Draft PR, Jira comment	The hook allows `gh pr create --draft` only with passing evidence	In Review
Learn	`learn`	Knowledge PR, `retro.md`	Lessons proposed, or "none" recorded	Done

4

What goes where

Most confusion about "rules versus skills versus plugins" disappears once you ask two questions about a piece of guidance: **must it always hold**, and **who needs it, when**. The layers below answer that, from the organisation down to a single developer.

Enterprise policy

`managed settings · MDM · team settings`

Which marketplaces and MCP servers are allowed, org-wide deny rules, no bypass mode, telemetry. Developers can't override it. `policy/managed-settings.json`

The standard

`plugin marketplace · this repo`

Always-on rules, skills, subagents, hooks, scripts and org connectors, versioned and installed into Claude Code and Cursor from one source. `plugins/`

The repo

`committed with the code`

Facts and wiring for one codebase: `AGENTS.md`, `.ai/repo.yaml`, data connectors in `.mcp.json`, `specs/`, schema snapshots, CI workflows. Written by `tools/aisdlc.py init`.

The developer

`personal, never committed`

Credentials in environment variables, local services (Docker Mongo, a local Airflow), personal preferences in user settings.

MECHANISM	PUT HERE	KEEP OUT	CLAUDE CODE	CURSOR	LOADED
Repo facts	What the repo is, its layout, commands, local conventions	Org rules, procedures, secrets	`CLAUDE.md` containing `@AGENTS.md`	`AGENTS.md` (also reads `CLAUDE.md`)	Every session
Repo manifest	Stack, check commands, test runner for given files, JUnit paths, protected paths, data env vars	Prose	`.ai/repo.yaml`: read by hooks, `evidence.py`, agents and CI		On demand
Rules	Short non-negotiables for every task: workflow contract, security, test standard (about 50 lines in total)	Procedures and examples	Plugins have no rules folder, so the SessionStart hook injects `rules/*.mdc`; repo-local rules go in `.claude/rules/` with `paths:`	Plugin `rules/*.mdc` (`alwaysApply`, `globs`); Team Rules	Every session, or on matching paths
Skills	Procedures and know-how for one kind of task: PRD review, spec, verify, Airflow conventions	Always-on rules, secrets	`skills/<name>/SKILL.md`, `/plugin:skill`, `paths:`, `allowed-tools`	Same `SKILL.md` (Agent Skills), `/skill`, `paths`	Description always; body on use
Subagents	Work that needs its own context and tool limits: analyst, investigator, builder, QA, reviewer	Work needing the main conversation	`agents/*.md`: `disallowedTools`, `model`, `memory`, `isolation`	`agents/*.md`: `readonly`, `model`	When delegated

MECHANISM	PUT HERE	KEEP OUT	CLAUDE CODE	CURSOR	LOADED
Hooks	What must always or never happen: no secrets, no force-push or merge, no code before spec approval, no PR without evidence; context injection	Advice; slow work	`hooks/hooks.json`: PreToolUse, SessionStart, Stop; exec form; `onFailure: block`	`hooks.json`: beforeShellExecution, preToolUse, beforeMCPExecution, stop; `failClosed`	Every matching event
Scripts	Deterministic logic shared by skills, hooks and CI	Judgement calls	`${CLAUDE_PLUGIN_ROOT}/scripts/`	Plugin `scripts/`	When called
Connectors (MCP)	Live systems: Confluence and Jira, Sentry, MongoDB, the schema DB, Airflow, a browser	What a CLI does better: `git`, `gh`, test runners	`.mcp.json` (org SaaS in the plugin, data in the repo)	`mcp.json`, `.cursor/mcp.json`	Server at start; tools on demand
Plugins	Distribution and versioning of everything above the repo layer	Per-repo facts	`.claude-plugin/`, marketplace, `enabledPlugins`, `dependencies`	`.cursor-plugin/`, team marketplace set to Required	Install and update
Settings and policy	Permissions, sandbox, allowed marketplaces and MCP servers, telemetry	Workflow logic	`.claude/settings.json`, managed settings	Team settings, MDM hooks, MCP allowlist	Always
CI workflows	Independent verification; automation triggers	The only copy of a rule (rules also run locally)	`claude-code-action@v1` with `plugins`	`agent -p` CLI	On GitHub events

Rules of thumb

- Must always hold: a **hook** or a **CI check**. Prose is advice; hooks are enforcement.
- Needed for some tasks: a **skill**, with `paths` if it concerns certain files.
- A fact about this repo: `AGENTS.md` for people, `.ai/repo.yaml` for machines.
- Live data: a **connector**, read-only, local or dev only.
- Must reach every developer and stay current: put it in the **plugin** and version it.
- A judgement call: a **subagent** with the right tools, then a human gate.

5

One builder, three repos

The builder is a single agent persona with an engineering discipline: test-first, small commits, stop when stuck. Its stack expertise comes from versioned packs that the repo selects, so it never has to remember three sets of conventions. A developer who mostly writes React can ship an Airflow change with the same guardrails, because the Airflow conventions, tests and checks arrive with the repo.

data-airflow
stack-airflow · `airflow-dag`

- Idempotent, interval-driven writes
- DagBag, idempotency and `dag.test()` tests
- Writes `payroll_run_status`

→ schema snapshot + `schema_snapshot.py diff`

payroll-mfe
stack-mfe + stack-frontend · `mfe-remote`

- Exposes `./StatusPanel`
- Component tests per AC
- Contract test of the exposed module

→ federation DTS types + consumer test

web-shell
stack-frontend · `frontend-feature`

- Route, role gate, flag
- Playwright per AC, screenshots, axe
- Fallback when the remote fails

How the specialisation happens

- **The manifest.** `.ai/repo.yaml` declares the stack, the check commands, the test runner for given files, JUnit paths and protected paths. Hooks, scripts, agents and CI all read the same file.
- **Session context.** At start, resume and after compaction, the sdlc-core SessionStart hook injects the stack, the checks, the branch's Jira key, the spec status and the evidence status, plus the always-on rules.
- **Path-activated stack skills.** `airflow-dag` declares `paths: dags/**, include/**`, so it loads when the builder touches DAG code, in Claude Code and in Cursor alike. `stack-mfe` depends on `stack-frontend`, so an MFE repo gets both.
- **The builder subagent** reads the manifest and loads the stack skill before every task. To make the builder the main session, start Claude Code with `claude --agent sdlc-core:builder`.

Stories that span repos

The spec's "Repo impact and sequencing" table sets the order: producers before consumers, each step backward compatible and safe to deploy alone. Each repo gets its own branch, evidence and draft PR, all linked to one Jira epic. The spec is approved once, in the lead repo; the other branches carry a copy with a `canonical:` link. `evidence.py` reads the spec's table to decide which ACs each repo must cover, so the MFE's PR is judged only on AC-1, 3, 4, 6 and 7.

6

The PRD gate: reporting gaps to the product owner

What is checked

Deterministically, with `prd_lint.py`, the same way every time:

- 15 sections, 9 of them required: summary, problem, goals, non-goals, users and roles, functional requirements, acceptance criteria, NFRs and open questions.

What the product owner receives

From the real run on the example PRD `PAY-412` (score 16/100, 3 blocking, 13 advisory):

- Placeholders (`TBD` , `<name>` , "to be decided"), empty sections, and N/A where it needs confirming.
- Acceptance criteria written as Given/When/Then, and every FR covered by an AC.
- Vague terms ("fast", "intuitive", "real-time") on lines without a number.
- NFRs about performance, freshness or availability without numbers.
- No error, empty or no-access states described.
- Unresolved open questions, which block unless tagged `[non-blocking]` .

With judgement, by the read-only `prd-analyst` subagent against a rubric: contradictions, roles and data scope, unhappy paths for UI and pipelines, data feasibility against schema snapshots, hidden cross-team scope, and known production incidents in the area.

```
## Questions for the product owner
1. (must answer) What is explicitly out of scope
   for this release?
2. (must answer) Please answer, or mark it
   [non-blocking]: 'Should admins of multi-
entity
   employers see all entities?'
3. (must answer) ... 'Which statuses exist?
   (Pending, Processing, Completed, Failed?)'
6. (please consider) What should users see when
data
   is empty, loading fails, or they lack access?
   Proposed default: Empty: neutral message;
   failure: retry option; no access: guidance
   without confidential details.
11. (please consider) Can you put a number on:
   'The page should load fast.'?
   Proposed default: p95 < 2 s ...
```

After the fixes, the same PRD scores 97/100 and is `READY` .

The loop

1. The product owner adds the label `prd-ready` . Confluence Automation's "Page labeled" trigger sends a web request to GitHub `repository_dispatch` (event `prd-ready` , payload with the page id and Jira key).
2. `ai-prd-intake.yml` fetches the page version as Markdown (`atlassian.py page-md`), lints it, and runs the `prd-review` skill headlessly. A `--json-schema` forces a typed verdict.
3. `NEEDS_CLARIFICATION` : the workflow adds a Confluence footer comment with the questions, creates a "Clarify PRD" Jira task for the product owner, and moves the epic to Needs Clarification.
4. `READY` : context and a draft spec are produced and opened as a draft spec PR for the tech lead.
5. A new page version re-runs the gate, and the new review lists what was "Resolved since version N".

`WHY` A product owner should get one consolidated, plain-language list with suggested defaults to accept, not a chain of chat threads. The clarification reasons are recorded, and they feed the PRD template and product-owner coaching (section 10).

7

Context sources: schemas, history and live systems

SOURCE	CONNECTOR	MODE	USED FOR
Confluence, Jira	Atlassian Rovo MCP `https://mcp.atlassian.com/v2/mcp` (OAuth); `atlassian.py` REST in CI (API token; Cloud and Data Center); `sooperset/mcp-atlassian` for Data Center	Read, plus comment, create task and transition	PRD fetch, product-owner feedback, Jira state, postmortems (CQL), bugs (JQL)
MongoDB	`mongodb-mcp-server@3.1.0 --readOnly --indexCheck`, telemetry off, at most 20 documents per query	Read-only, local or dev	`collection-schema`, indexes, counts, explain
Schema database	`@bytebase/dbhub@1.5.0` with TOML: `execute_sql` has `readonly = true`, `max_rows = 200`; plus a SELECT-only DB user	Read-only	Tables, columns and schema-registry lookups
Airflow	`astro-airflow-mcp@0.9.1` with `AF_READ_ONLY=true`; REST `/api/v2` as a fallback	Read-only	Failed runs, import errors, task logs
Production errors	Sentry MCP `https://mcp.sentry.dev/mcp` (Datadog, Grafana and PagerDuty are alternatives)	Read	Errors on the routes and components a story touches
Code and history	`git`, `gh`, `rg` (CLIs, not MCP)	Read	Similar features, `git log -S`, reverts, earlier PRs
Browser	`@playwright/mcp@0.0.83 --headless --isolated`	Local app only	Exploring UI states while building; evidence comes from Playwright tests
Team knowledge	`knowledge/` and each repo's `docs/knowledge/` (git)	Read	Reviewed lessons, cited by id

Rules that keep this safe

- **Snapshots first.** Committed JSON Schema snapshots in `docs/schemas/` are reviewed and diffable. Live MCP covers only what they lack. `schema_snapshot.py` samples to a file, so record values never enter the conversation, and `diff` exits 1 on a breaking change.
- **Read-only at two levels**: the server flag (`--readOnly`, `readonly = true`, `AF_READ_ONLY`) and a read-only database user. The shell hook also blocks writes through `mongosh` and `psql`.
- **Untrusted content.** PRDs, tickets, logs and rows are data, never instructions. This addresses the "lethal trifecta" (private data, untrusted content and a way out) and Meta's "Rule of Two": the research stages read; the build stage writes code but has no outbound channel except `git push` to its own branch; PR creation is gated; a human merges.
- **Prefer CLIs** for `git`, `gh` and test runners. They cost less context and behave deterministically. Thoughtworks lists "MCP by default" under Caution.

8

Specs that drive the build

The PRD says what and why; the spec says how, precisely enough that build, verify and review can be checked against it. It lives in git as `specs/<KEY>/spec.md` and is reviewed like code.

- **Traceable IDs.** `FR-n` requirements from the PRD; `AC-n` criteria in Given/When/Then, each citing its FRs.

- **Design only where something changes**: data (before and after, indexes, compatibility), pipeline tasks, contracts (API, exposed modules, events), UI states.
- **A repo impact and sequencing table** that tooling reads: order, repo, change, ACs, backward compatibility.
- **A test plan** giving the level, the qualified test name (`PAY-412 AC-3: ...`) and the repo for every AC.
- **A risk tier**: low, medium, or high (auth, PII, migrations, money). A high tier lists the extra approvals it needs.
- **Open questions must read "None."** before approval.

Approval is human and enforced. The edit hook refuses any agent edit that sets `status: approved`, and refuses code changes on a branch whose spec is still a draft. The eval suite measured the difference: asked to implement against a draft spec, the agent stopped and explained the approval step with the plugin (score 1.0), and wrote the code anyway without it (0.5).

> `FAST LANE` Spec-driven development is criticised for applying one heavy process to every change (Böckeler turned a small bug into 4 user stories and 16 ACs). Low-risk changes in one repo with no data, contract or auth impact use a short spec: summary, ACs, repo impact and test plan. Ceremony scales with risk; the gates don't.

9

Credible evidence: how tests prove the change

An agent saying "all tests pass" is not evidence. Green CI isn't enough either: public benchmarks found that 15–28% of "passing" patches were wrong because the tests were weak, and models under pressure modify tests to pass. The protocol below makes the claim checkable.

LEVEL	WHAT IT IS	ACCEPTED FOR
L0 Claimed	The agent says it works	Nothing
L1 Local	`evidence.py run` at a committed SHA: checks with logs and SHA-256 hashes, AC coverage, fail-to-pass	Opening a draft PR (the hook checks it)
L2 CI	The same script on a clean runner at the PR head SHA, plus changed-line coverage; artifacts uploaded; sticky PR comment	Review and merge (a required check)
L3 Attested	The L2 artifacts signed with `actions/attest@v4` (Sigstore provenance bound to commit and workflow)	Audit and regulated changes

The protocol

1. **Tests cite acceptance criteria by qualified ID:** `it("PAY-412 AC-3: ...")` or `test_pay_412_ac_3_...`. `ac_coverage.py` reads JUnit from pytest, Vitest, Jest and Playwright, and marks each AC covered, failing, manual or missing. The Jira key keeps AC-3 of one story from matching another's.
2. **Fail-to-pass**, after SWE-bench: `fail_to_pass.py` copies the new tests onto the merge-base in a temporary worktree. Each must fail there and pass on the branch. A test that passes on both doesn't prove the change; one that only errors on import counts as weak.
3. **No silent weakening.** The edit hook refuses new `skip`, `only`, `xfail` and `fixme` markers unless the line carries a justification, and refuses hand edits to `.ai/evidence/`.
4. **An independent check.** The read-only `qa-verifier` judges whether each AC test asserts the outcome, avoids tautologies, doesn't mock the unit under test, and covers unhappy paths.
5. **Binding to code.** The manifest records the head SHA, the merge-base, the spec's hash and status, the runner and the CI run URL. The PR gate rejects stale evidence (HEAD moved) and dirty trees.

What each stack contributes

STACK	REQUIRED EVIDENCE	TYPICAL CHECKS IN `.AI/REPO.YAML`
Airflow	DagBag has no import errors; owner, tags and retries everywhere; no import-time `now()` or `Variable.get()`; an idempotency test per write path; `dag.test()` on local services; contract snapshot	`ruff check`, `pytest` to JUnit and coverage, `airflow dags list-import-errors`
MFE	Component tests per AC; a contract test of the exposed module; DTS type build with `abortOnError`; a federation smoke test with fallback	`npm run lint`, `tsc --noEmit`, `vitest` to JUnit, build, Playwright
Frontend	Testing Library per AC with MSW; Playwright per cross-route AC with screenshots, traces and an axe scan; both flag states	Same as the MFE row

What reviewers see (real output from `make demo`)

```
## Evidence: PASS
- Commit `c7e8b81fc419` on `feat/PAY-412-status` (base `ac95c113df4f`)
- Spec: `specs/PAY-412/spec.md` (status: approved, sha256 `082894aea488`)
| Check | Result | Time  | Log sha256     |
| unit  | pass   | 0.07s | `a4f11338b245` |
### Acceptance-criteria coverage: PAY-412 (PASS)
| AC-2 | covered | `test_pay_412_ac_2_aliases_map_to_completed` (passed) |
### Fail-to-pass: PASS
| tests.test_status::test_pay_412_ac_2_aliases_map_to_completed | failed | passed | FAIL_TO_PASS (strong)
|
| tests.test_status::test_upper                                 | passed | passed | PASS_TO_PASS
|
```

10

Compounding knowledge

Every story should make the next one easier. The `learn` skill runs after review or merge. It collects review comments, rework commits, CI failures, PRD clarification rounds and linked incidents, and proposes changes through a reviewed PR. A lesson moves up a ladder each time it recurs; each rung enforces it more strictly.

RUNG 0

Observation

A review comment, a CI failure, an incident, a PRD clarification

RUNG 1

Knowledge entry

Reviewed, owned and dated; agents find it when it's relevant (`knowledge/`)

RUNG 2

Rule or skill text

Loaded every session, or on matching paths

RUNG 3

Hook, lint or test

Enforced deterministically. If a reviewer has said it twice, it belongs here.

LESSON TYPE	DESTINATION	OWNER
Fact about one repo	`AGENTS.md` or `docs/knowledge/` in that repo	Repo CODEOWNERS
Stack pitfall or pattern	`knowledge/<stack>/`, then the stack skill	Stack guild
Workflow gap	`sdlc-core` skills or rubric	Platform team
PRD gap	`standards/prd-template.md`, the rubric, `prd_lint.py`	Product ops

The seed entries in `knowledge/` show each rung. `KB-airflow-0001` (interval-based idempotent writes) was promoted to the Airflow skill. `KB-frontend-0001` (no request without access) was promoted to a security rule. `KB-process-0001` (PRDs must state unhappy paths) was promoted to lint rule `PRD-050`. Every entry has a `review_by` date; a weekly job runs `aisdlc.py knowledge-audit` and opens an issue for stale entries.

BOTH SIDES LEARN The product side compounds too. Each story's `prd-review.json` records why the PRD bounced, and the most common reasons become template prompts and product-owner coaching. This is the same idea as Block's "turn repeated model failures into linters" and Every's compound-engineering step, applied to requirements as well as code.

11

Setups to try

Six setups, from the cheapest to the most automated. They share the same plugin, scripts and repo files, so moving from one to the next adds automation without changing the process.

A Cursor, team marketplace

BEST FOR	Teams that live in Cursor; the first two weeks
SET UP	Import this repo as a Cursor team marketplace and set `sdlc-core` and `sdlc-connectors` to Required; run `aisdlc.py init` in each repo
YOU GET	Always-on rules, `/prd-review` to `/verify`, read-only subagents, shell, edit and PR hooks, Atlassian and Sentry MCP
WATCH FOR	`preToolUse` can't ask, so it denies with a note; confirm plugin hook paths on your version
MOVE ON	When the PRD gate and evidence flow have run on 3 stories

B Claude Code with the plugin START HERE

BEST FOR	The pilot; anyone on Claude Code (CLI, desktop or IDE extension)
SET UP	`aisdlc.py init` writes the marketplace and plugins into `.claude/settings.json`; developers trust the folder and the plugins install; `/mcp` authenticates Atlassian and Sentry
YOU GET	`/sdlc-core:prd-to-pr` end to end, every hook including session context and the verify reminder, and evals
WATCH FOR	`python3` and `gh` on PATH; PyYAML for `evidence.py`
MOVE ON	After 10 to 20 stories, with metrics compared against the baseline

C Hybrid: Cursor plus Claude Code

BEST FOR	Teams split between the tools
SET UP	A and B together on the same repo files; run Claude Code in Cursor's terminal or as the IDE extension
YOU GET	The same skills, rules, hooks and evidence whichever tool a developer opens
WATCH FOR	With Cursor's third-party import on and Claude plugins installed, some hooks run twice (harmless)
MOVE ON	This is a steady state for mixed teams

D CI-driven intake, build and evidence

BEST FOR	Making the flow team-wide and auditable
SET UP	The `ai-prd-intake`, `ai-build`, `ai-evidence`, `ai-review` and `ai-learn` workflows (init adds them); secrets for Anthropic (or OIDC federation) and Atlassian; Confluence Automation "Page labeled" → `repository_dispatch`; make `ai-evidence` a required check
YOU GET	The PRD gate on a label, draft spec PRs, headless builds to draft PRs, attested evidence on every PR, learning PRs after merge
WATCH FOR	A private marketplace needs a read token in CI (`gh auth setup-git`)
MOVE ON	When the required check is stable and review time per PR isn't rising

E Parallel cloud agents

BEST FOR	Running several stories at once from the web, phone or Slack
SET UP	`aisdlc.py vendor`, because Claude Code cloud sessions don't install repo-declared plugins and Cursor cloud agents run only `.cursor/hooks.json`; a setup script or `.cursor/environment.json` installs the toolchain
YOU GET	The same skills and hooks inside the cloud sandbox; draft PRs with evidence
WATCH FOR	Data connectors need reachable dev databases, or snapshots only; re-run `vendor` when the standard updates
MOVE ON	When review capacity, not agent capacity, becomes the limit

F Orchestrated at scale

BEST FOR	Dozens of stories a week, cross-repo releases, a custom approval UI
SET UP	Claude Agent SDK (`plugins`, `setting_sources`) driven by a durable workflow engine, or Claude Code workflows; reuse the same plugin and scripts
YOU GET	The draft plan's target architecture, built from proven parts
WATCH FOR	Building it before A to E have proved the process
MOVE ON	This is the end state

```
# one-time per repo (from a checkout of the standard)
python3 tools/aisdlc.py init   --target ../data-airflow --stack airflow  --jira PAY --description "Payroll
pipelines"
python3 tools/aisdlc.py init   --target ../payroll-mfe  --stack mfe      --jira PAY
python3 tools/aisdlc.py init   --target ../web-shell    --stack frontend --jira PAY
python3 tools/aisdlc.py doctor --target ../data-airflow
# then, in the repo, with Claude Code
/sdlc-core:prd-to-pr https://acme.atlassian.net/wiki/spaces/PAY/pages/123456789
```

12

Claude Code features and how the standard uses them

FEATURE	HOW THE STANDARD USES IT	WHERE
Memory files	`CLAUDE.md` imports `@AGENTS.md` so both tools share one file; path-scoped `.claude/rules/` for repo-local rules	`templates/repo/common/`
Skills	14 skills; `paths` loads stack skills automatically; `${CLAUDE_PLUGIN_ROOT}` in both the body and `allowed-tools` runs bundled scripts without prompts; `disable-model-invocation` on the orchestrator	`plugins/*/skills/`
Subagents	Five agents; `disallowedTools` makes four of them read-only (plugin agents ignore `permissionMode` and `hooks`, so tool lists do the enforcing); optional `memory: project` and `isolation: worktree`	`plugins/sdlc-core/agents/`
Hooks	SessionStart for context and rules; PreToolUse for the shell, edit and MCP PR guards; Stop for the verify reminder (loop-guarded); exec form; `onFailure: block` fails closed	`hooks/hooks.json`
Plugins and marketplace	Plugin `dependencies` (stack-mfe → stack-frontend → sdlc-core); per-repo `extraKnownMarketplaces` + `enabledPlugins`; `autoUpdate`; `userConfig` is available for org-specific values	`.claude-plugin/`
Plugin CLI	`validate --strict` in CI; `details` for the token budget; `eval` with a no-plugin baseline; `tag` for releases	Makefile, `validate.yml`
MCP	OAuth SaaS servers in the plugin's `.mcp.json`; data servers in the repo's `.mcp.json` with `${VAR:-default}`; tool search loads tool schemas on demand	`sdlc-connectors`, templates
Permissions and sandbox	Repo allow, ask and deny lists; OS sandbox for Bash (bubblewrap on Linux, Seatbelt on macOS); `disableBypassPermissionsMode`	`.claude/settings.json`, `policy/`
Managed settings	Force-enable the core plugins, `strictKnownMarketplaces`, `allowManagedMcpServersOnly`, OpenTelemetry env, `companyAnnouncements`; optionally `strictPluginOnlyCustomization`	`policy/managed-settings.json`
Headless mode	`claude -p` with `--json-schema` for typed verdicts, `--max-turns`, `--allowedTools`, `--plugin-dir`	CI workflows
GitHub Action	`anthropics/claude-code-action@v1` with the `plugins` and `plugin_marketplaces` inputs, the `structured_output` output, and workload identity federation (no long-lived key)	`templates/repo/common/.github/`
Plugin seeding	`CLAUDE_CODE_PLUGIN_SEED_DIR` bakes plugins into CI images that can't clone at runtime	Setup D and F
Agent SDK	Python `claude-agent-sdk`, TypeScript `@anthropic-ai/claude-agent-sdk`; `plugins` and `setting_sources` reuse the same standard	Setup F
Cloud sessions, Remote Control	Parallel stories in Anthropic-hosted sandboxes; they load the repo's `.claude/`, so use vendored mode	Setup E
Scheduled tasks, `/loop`	Nightly knowledge audit, flaky-test triage, re-vendoring checks (cloud scheduling is a research preview)	Optional
Review and security	`/code-review`, `/security-review`; the managed Claude Code Review reads `REVIEW.md` and never approves; the Claude Security plugin writes SARIF	`REVIEW.md` template
Plan mode, checkpoints	Plan mode for spec work without edits; `/rewind` undoes a bad build attempt (file edits only, not Bash side effects)	Interactive use
Telemetry and analytics	OTel metrics `claude_code.cost.usage`, `token.usage`, `session.count`; `plugin_loaded` events; the plugin analytics API shows adoption	Managed settings
Workflows	Script-orchestrated subagents can make `prd-to-pr` a deterministic "blueprint" (the Stripe pattern) once the skills are stable	Setup F candidate

FEATURE	HOW THE STANDARD USES IT	WHERE
Auto memory	Personal notes per developer; team knowledge stays in git where it's reviewed	Default on

13

Cursor features and how the standard uses them

FEATURE	HOW THE STANDARD USES IT	WHERE
Rules	`.mdc` with `alwaysApply` or `globs` from the plugin's `rules/`; Team Rules can enforce them org-wide	`plugins/sdlc-core/rules/`
Agent Skills (2.4+)	The same `SKILL.md` files; `paths` scoping; `/skill-name`; also read from `.claude/skills/` in vendored repos	`plugins/*/skills/`
Subagents	The same `agents/*.md`; `readonly: true` enforces read-only roles; also read from `.claude/agents/`	`plugins/sdlc-core/agents/`
Hooks	`beforeShellExecution`, `preToolUse` (Write), `beforeMCPExecution`, `sessionStart` (`additional_context`), `stop` (`followup_message`); `failClosed`; MDM or dashboard distribution on Enterprise	`hooks/cursor-hooks.json`
Plugins, team marketplaces	`.cursor-plugin/plugin.json` beside the Claude manifest; Required install mode; group-based access	`.cursor-plugin/`
MCP	`mcp.json` in the plugin; `.cursor/mcp.json` in repos with `${env:VAR}`; enterprise MCP allowlist	Templates
Bugbot	Reads `.cursor/BUGBOT.md` (the review guidance, same text as `REVIEW.md`); project rules don't apply to Bugbot	`templates/repo/common/.cursor/`
Cloud agents, Automations	`.cursor/environment.json` installs the toolchain; triggers from Slack, GitHub, Linear and Jira; vendored hooks give guardrails	Setup E
CLI	`agent -p --output-format json`, `--force` for edits; let deterministic steps run `git`, `gh` and `evidence.py`	Setup D alternative
AGENTS.md, CLAUDE.md	Read natively, including nested files	Every repo
Enterprise controls	Privacy Mode, audit logs, model access control, terminal sandbox, repository blocklist	Dashboard

Details and a two-minute check for your Cursor version are in `cursor/README.md`.

14

Running it in an enterprise

Distribution and updates

CONSUMER	HOW A NEW VERSION ARRIVES
Claude Code, plugin mode	`autoUpdate: true` on the marketplace (repo or managed settings), or `/plugin update`; running sessions need `/reload-plugins`
Cursor team marketplace	Auto Refresh (needs the GitHub App) or a manual refresh in the dashboard
Vendored repos	Re-run `aisdlc.py vendor`; review the diff in a PR; the version is pinned in `.claude/aisdlc/VERSION.json`
CI workflows	Bump `AISDLC_REF` to the new release tag

Each plugin is versioned with semver. Patch releases are wording; minor releases add a skill or check; major releases change a gate. `tests/test_manifests.py` fails the build if the Claude manifest, the Cursor manifest and the marketplace disagree. `claude plugin tag` cuts the release. CODEOWNERS give each stack pack to its guild.

Security model

- **Least privilege by stage.** Analysts and reviewers are read-only. Connectors are read-only against local or dev data. The builder writes only to its own branch. PR creation is gated. Only people merge.
- **Untrusted input stays data.** PRDs, tickets, logs and records never become instructions, and no single stage holds private data, untrusted content and an outbound channel together.
- **Deterministic floor.** Hooks block secret reads and writes, force-pushes, merges, infrastructure and cluster changes, writes to data stores, `curl | sh`, and environment dumps. Claude Code's guards fail closed.
- **Org policy.** Managed settings allow only this marketplace and the vetted MCP server commands and URLs, forbid bypass mode, and export telemetry.
- **Audit trail.** OpenTelemetry events, the Atlassian MCP audit log, git history of specs and evidence, CI run links, and artifact attestations.

Roles

ROLE	OWNS
Product owner	The PRD; answers gate questions or accepts the suggested defaults
Tech lead	Spec approval and risk tier; cross-repo sequencing
Developer	Runs the workflow, reviews the AI PR before requesting reviewers, and merges with code owners
Platform team	`sdlc-core`, connectors, CI templates, metrics, releases
Stack guilds	Their stack pack and knowledge entries
Security	Managed policy, the connector allowlist, high-risk approvals
QA	Test strategy, evidence validity, eval cases

Cost

Measured with `claude plugin details`, the always-on context cost is about 1.6k tokens for `sdlc-core`, 0.3k for the connectors, and 0.15–0.18k per stack pack, plus about 0.6k of injected rules. Skill and agent bodies load only when used. A behavioural eval case cost about $0.10 per run. Track cost per merged PR with the OTel metric `claude_code.cost.usage`.

15

Rollout and metrics

WEEKS	DO	EXIT CRITERIA
0–1	Pick one repo per stack. Baseline the last 20 comparable tickets (lead time, review cycles, CI failures, reopen rate). Run `aisdlc.py init` and `doctor`. Publish the PRD template. Apply managed settings to the pilot group.	`doctor` passes; the evidence workflow runs on an ordinary PR
2–3	Run the PRD gate on every new PRD (Setup B plus the intake workflow). Write specs for 3 stories.	At least one PRD bounced and fixed in one round; 3 approved specs

WEEKS	DO	EXIT CRITERIA
3–5	Build and verify 5–10 low- and medium-risk stories (Setups A, B and C). Make `ai-evidence` a required check. Run `learn` after each merge.	Evidence PASS on every draft PR; no gate bypassed; the first knowledge PRs merged
6–8	Add CI-driven intake and build (D), and cloud agents (E) if wanted. Run evals in CI for skill changes.	Go/no-go on the metrics below

METRIC	MEASURED FROM	PILOT TARGET (AGREE BEFORE STARTING)
PRD first-pass readiness	`prd-review.json` verdicts and reasons	Rising; top reasons feed the template
READY to draft PR lead time	Jira timestamps, `retro.md`	Below the baseline median
AC automation	Evidence manifest (covered vs manual)	90% or more of in-scope ACs
Fail-to-pass rate	Evidence manifest	90% or more of behaviour PRs have a strong FAIL_TO_PASS
Review burden	GitHub: review minutes, change-request cycles	No higher than baseline
Rework	Git: share of AI lines rewritten in review	Tracked and falling
Escaped defects	Jira bugs linked within 30 days	No higher than baseline
Cost per merged PR	OTel `claude_code.cost.usage`	Tracked; alert on outliers
Gate integrity	Branch protection, hook denials	Zero unreviewed merges

DON'T Don't use "percent of code written by AI" or PR counts as targets. DORA 2025 found that AI raises throughput and instability together. Faros saw PRs grow 154% and review time 91%. Google's migration work measures end-to-end time.

16

What others do

A summary of the sourced survey in `docs/research.md`, which has dates, numbers and links.

ORGANISATION	WHAT THEY DO	LESSON WE ADOPT
Stripe	Minions: one-shot agents in pre-warmed devboxes, "blueprints" mixing deterministic and agent steps, curated MCP tools. Over 1,300 merged PRs a week, all human-reviewed	Deterministic skeleton; at most 2 CI rounds, then a human; rules shared between Cursor and Claude Code
Spotify	Honk on the Claude Agent SDK; verifiers in the stop hook; an LLM judge. Merged changes rose from 8,100 to 17,000 a year, faster than "some of our verification controls could adapt"	Verify before a PR exists; expect PRs that pass CI but are wrong; data repos need tests before agents
Uber	uReview on over 90% of diffs; AutoCover with a mutation-testing validator; design-to-draft-PR agent	Validate tests objectively; precision over volume in AI review
Meta	TestGen-LLM filters (builds, passes, not flaky, adds coverage); RADAR risk-gated review of low-risk diffs	Filter AI-written tests by objective criteria; tier review by risk
Google	AST and static analysis plus LLM edits for migrations; review speed was the bottleneck	Measure end-to-end time; cap change volume to what reviewers absorb
Ramp	Inspect: a full-stack VM per session, screenshots for frontend work, PR opened with the requester's token	Give agents full context; the requester can't self-approve
Block	goose; an 11k-file UI-library migration with a language-service tool, 50+ hint files and 20+ linters	Turn repeated model failures into linters
Airbnb	3.5k test files migrated in 6 weeks with a per-file state machine and retry caps	Retries and context beat prompt polish
GitHub, Atlassian	Issue or Jira item → sandbox → draft PR; review against Jira acceptance criteria	Draft PRs; review against the ACs
Every, Amazon (Kiro), GitHub (Spec Kit)	Compound engineering; spec-driven requirements → design → tasks, with tests from specs	Spec first with checkable ends; a compounding step after each story

What the data says

- **DORA 2025** (about 5,000 respondents): 90% use AI; it amplifies existing strengths and weaknesses. Small batches, quality internal platforms and AI-accessible internal data predict good outcomes.
- **METR**: experienced developers were 19% slower in 2025 while believing they were faster; the 2026 update shows −18% time (CI −38% to +9%). Measure against a baseline.
- **Faros AI** (vendor): more PRs and much longer reviews; incidents per PR rose in 2026 Q2. Review is the bottleneck.
- **CodeRabbit** (vendor, 470 PRs): AI-co-authored PRs had about 1.7x more issues. An independent review is needed.
- **SWE-bench, UTBoost, ImpossibleBench**: validation needs fail-to-pass tests; weak tests pass wrong patches; models modify tests to pass.

17

What doesn't work

- **Building the orchestrator first.** It means months of infrastructure before learning whether specs, gates and evidence work for your teams.
- **One agent, one giant prompt, no gates.** Agents drift from specs and overreach (Spotify, Böckeler). Use a deterministic skeleton with agent steps.
- **Prose-only rules for things that must hold.** Use hooks and CI checks. The spec-gate eval shows the difference.
- **Letting agents change tests to get green.** Block skip markers, require fail-to-pass, and have a separate reviewer check test semantics.
- **Unbounded retries.** Cap them (3 per task, 2 verify rounds) and escalate with what was tried.
- **Optimising for "% AI code" or PR count.** Instability and review time rise. Measure lead time, rework and escaped defects.
- **Large PRs.** Keep one story per branch, about 200-line tasks, and a PR per repo.
- **MCP for everything; ever-growing instructions.** Use CLIs for local tools, keep always-on rules near 50 lines, and put the rest in path-scoped skills.
- **Production data in prompts.** Use snapshots, read-only dev databases and synthetic fixtures.
- **Auto-merge before the pilot proves quality.** Even Meta's RADAR auto-lands only pre-selected low-risk diffs after ML risk scoring.
- **Skipping the baseline.** Perceived speed-ups differ from measured ones (METR).

18

What was verified

Verified in this repository (22 Oct 2026)

- `PASS` 77 unit tests on Python 3.9 and 3.13. They cover the PRD lint, AC coverage, fail-to-pass in throwaway git repos, evidence manifests, the Atlassian client against recorded requests (Cloud and Data Center), schema snapshots, every hook in both output formats, the onboarding CLI, and manifest consistency.
- `PASS` `claude plugin validate --strict` for the marketplace and all five plugins (Claude Code 2.1.308).
- `PASS` 3 behavioural evals with real model runs; the spec gate scored +0.5 against the no-plugin baseline.
- `PASS` `make demo`: PRD gate, cross-stack AC coverage, fail-to-pass evidence, and the PR gate going from deny to allow.

Check in your environment

- `TODO` Live calls to your Confluence, Jira, Sentry, MongoDB and schema database.
- `TODO` The GitHub workflows on real runners. They parse, but were not executed here.
- `TODO` Cursor running the plugin hooks (Cursor isn't available here): expect a blocked `git push --force`.
- `TODO` Claude Code cloud sessions with vendored configuration.
- `TODO` Replace the placeholder CODEOWNERS teams and the example Jira statuses with your own.

- `PASS` Formats checked against current documentation for Claude Code, Cursor, Atlassian, MongoDB, DBHub, Airflow 3, Playwright, and GitHub Action versions.

```
make test       # unit tests
make validate   # strict plugin validation
make demo       # deterministic end-to-end demo
make evals      # behavioural evals (costs money)
make pdf        # render this playbook to PDF
```

PRD to PR Playbook, version 1.0. Source: `docs/playbook.html` in the standard repository; render the PDF with `make pdf`. Research sources: `docs/research.md`. Eval results: `docs/evals.md`.